

Organization of Digital Computer Lab EECS112L

Lab #: Lab 4
Student Name: David Kim
Student ID#: XXXXXXXXXX
Date: 01/30/2026

1 Objective

The objective of this lab was to implement pipelining into the processor that we made in lab 2. This was done by implementing the IF_pipe_stage, ID_pipe_stage, EX_pipe_stage, EX_Forwarding_unit, Hazard_detection, and the mips_32 modules, as well as any registers for sending and receiving signals across pipeline stages.

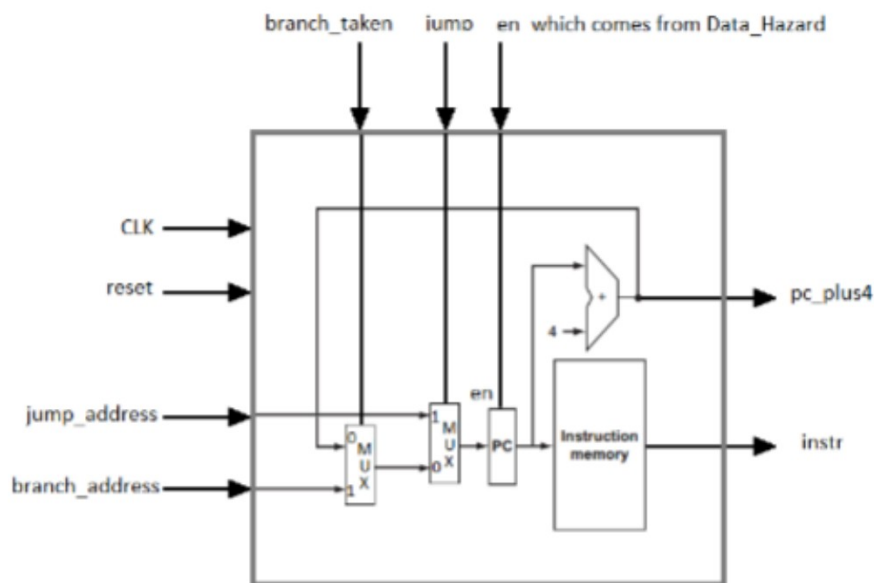
2 Procedure

IF_pipe_stage:

The IF_pipe_stage module was implemented using the block diagram provided, which shows the input, outputs, internal connections, and any modules that the IF pipe stage required.

This pipe stage instantiated

- 2 mux2 units
- 1 instruction memory unit



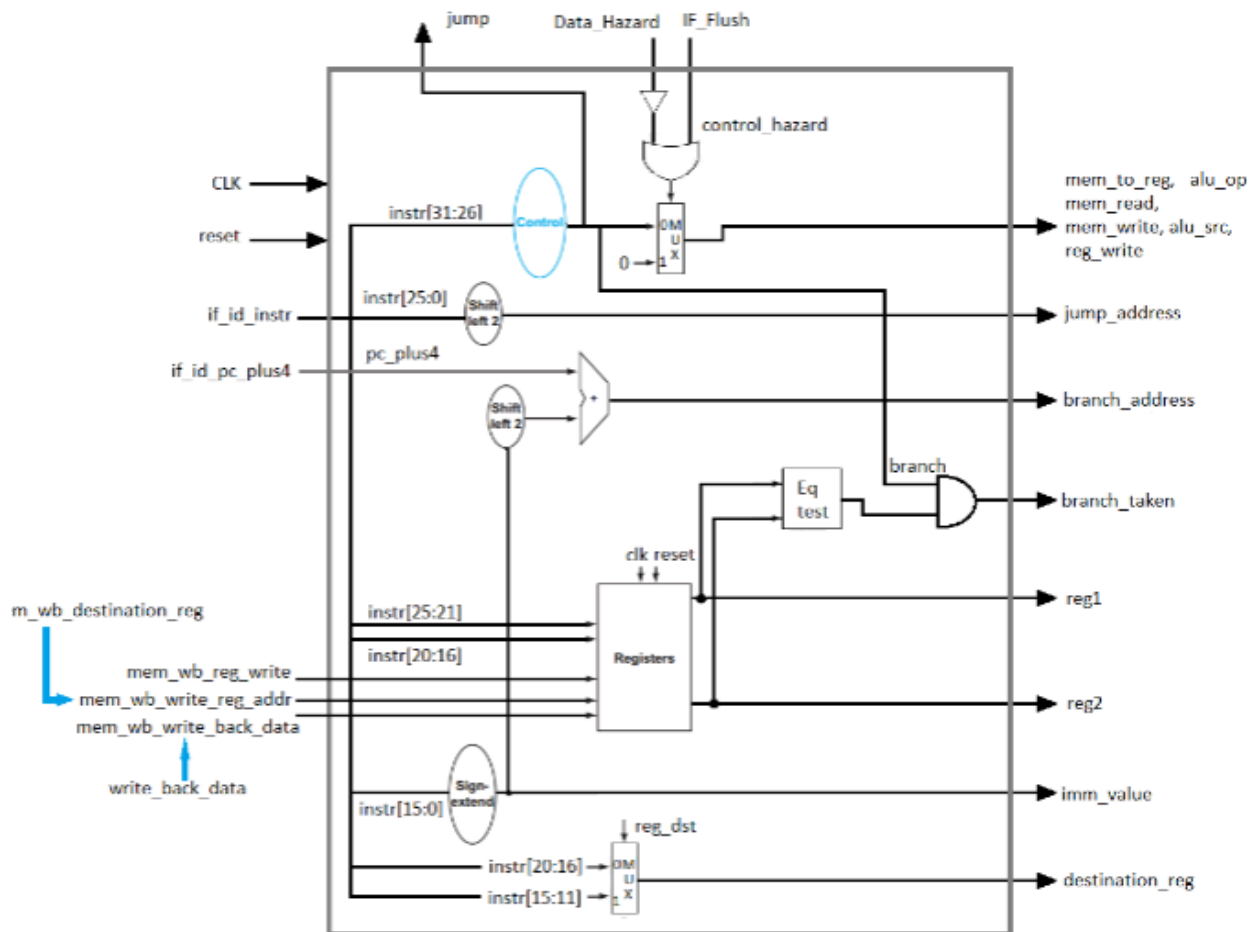
ID_pipe_stage:

The ID_pipe_stage module was implemented using the block diagram provided, which shows the input, outputs, internal connections, and any modules that the ID pipe stage required.

This pipe stage instantiated

- 2 mux2 units
- 1 sign extend unit
- 1 regfile unit
- 1 control unit

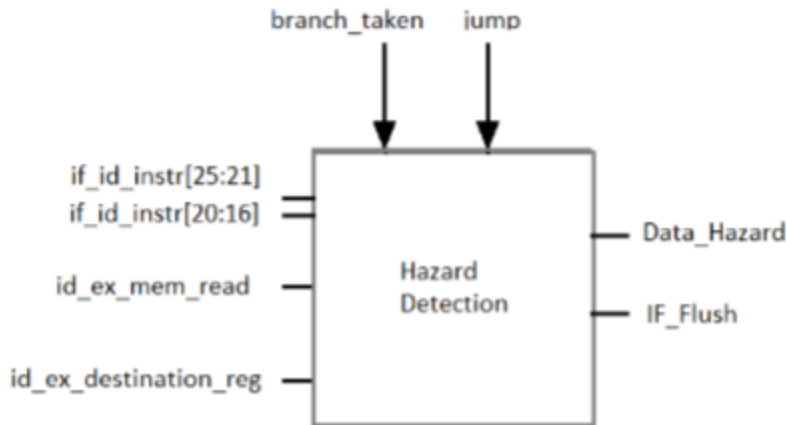
Figure 4: Instruction Decode.



Hazard Detection:

The Hazard_detection module was implemented using the block diagram provided, which shows the input and outputs.

The logic for this module was provided.

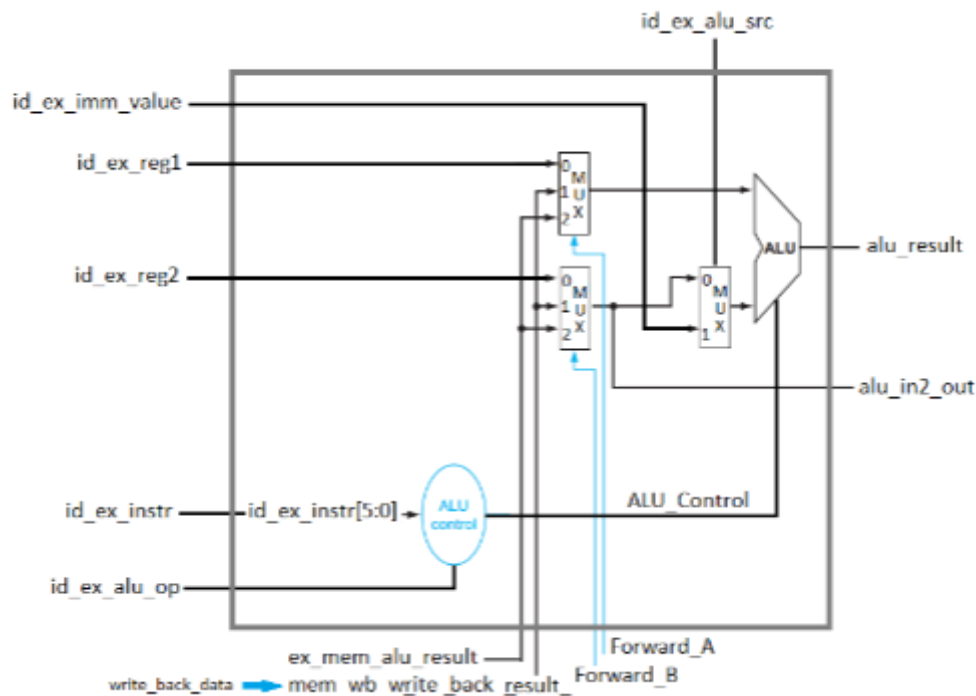


EX_pipe_stage:

The EX_pipe_stage module was implemented using the block diagram provided, which shows the input, outputs, internal connections, and any modules that the EX pipe stage required.

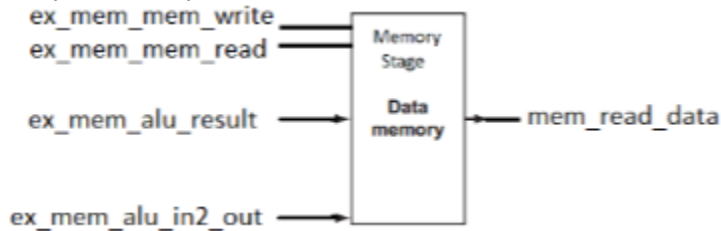
This pipe stage instantiated

- 2 mux4 units
- 1 mux2 unit
- 1 ALU unit
- 1 ALU control unit



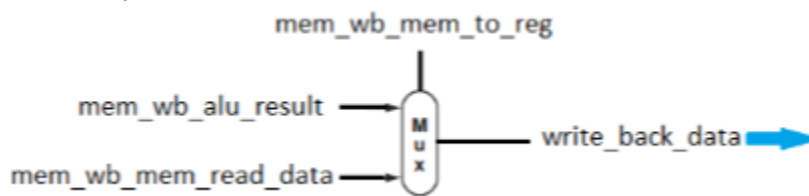
Data Memory:

Data Memory was implemented in the mips_32 module using the block diagram provided, which shows the input and outputs.



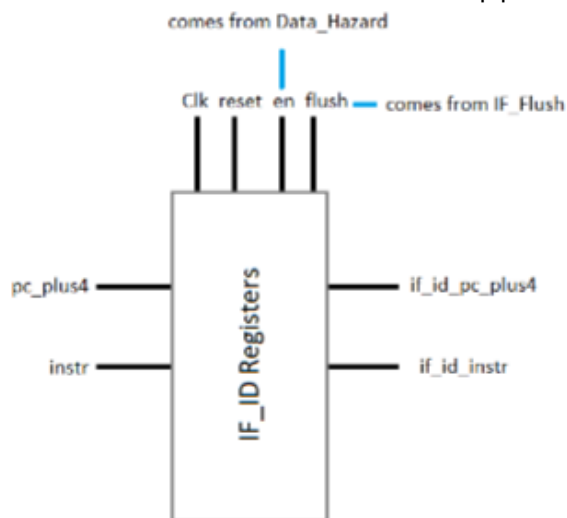
Write Back:

Write back was implemented in the mips_32 module using the block diagram provided, which shows the input and outputs to a mux2.



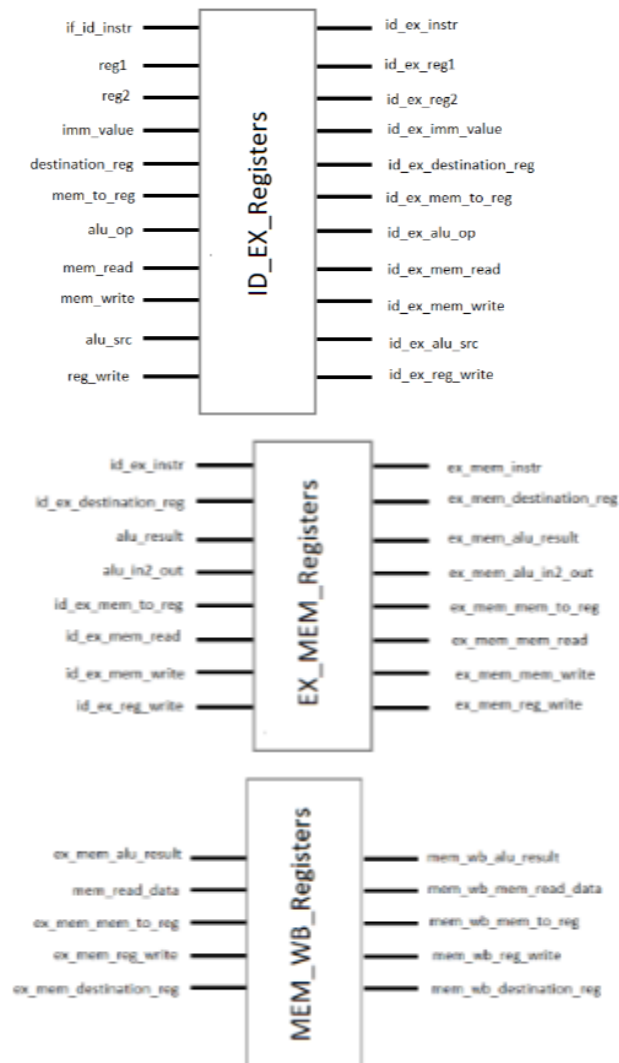
Registers:

IF_ID registers were implemented using pipe_reg_en, which required an en signal and a flush signal in order to send the data from the IF to the ID pipe stage.



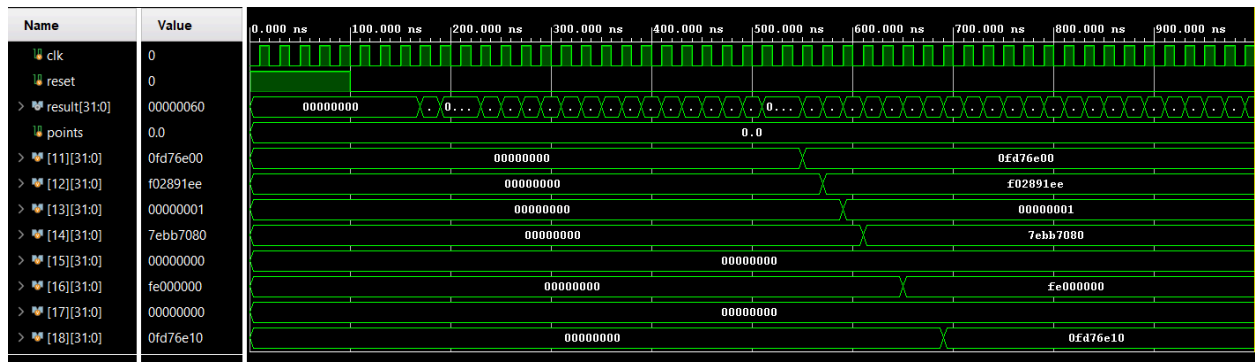
The remaining register used the pipe_reg module to send the data from one pipe stage to another. This module differed from the pipe_reg_en module as it didn't require an en signal to send the data, instead sending it unconditionally.

Figure 5: ID/EX Registers.



3 Simulation Results

The test cases from the provided testbench module were used to check for correctness of the implementation.



This is a waveform of the input and outputs from 0 to 1000 ns. Since there is an initial 100 ns delay during which reset = 1, the test cases start at 100 ns.

The 8 variables that are tracked after points in the waveform are the data memory locations from 11 to 19. As the simulation continues, the locations in the data memory get updated one by one, achieving their expected results.

All provided test cases passed as well.